

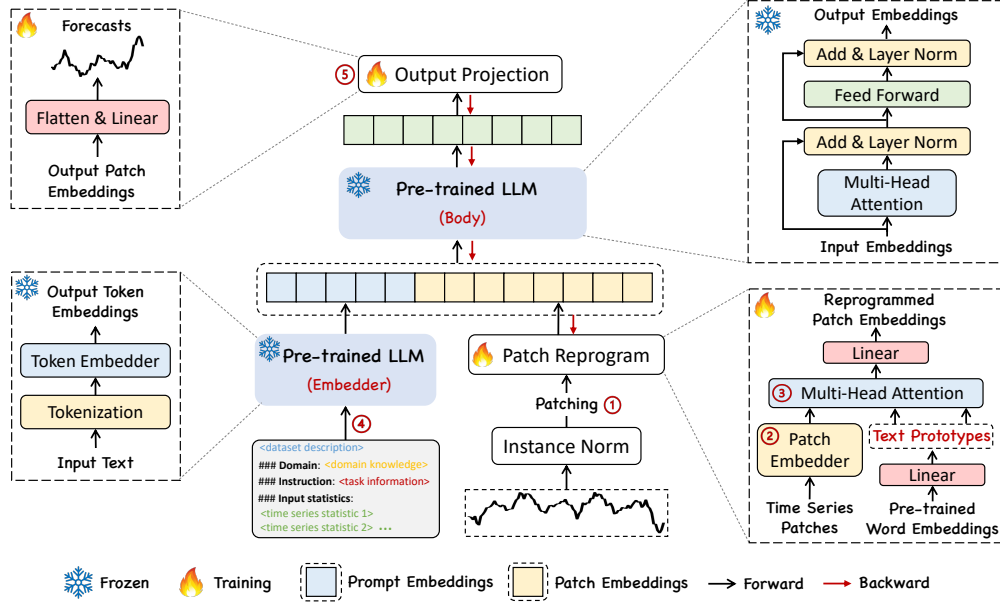


Figure 2: The model framework of TIME-LLM. Given an input time series, we first tokenize and embed it via ① patching along with a ② customized embedding layer. ③ These patch embeddings are then reprogrammed with condensed text prototypes to align two modalities. To augment the LLM’s reasoning ability, ④ additional prompt prefixes are added to the input to direct the transformation of input patches. ⑤ The output patches from the LLM are projected to generate the forecasts.

univariate time series, which are subsequently processed independently (Nie et al., 2023). The i -th series is denoted as $\mathbf{X}^{(i)} \in \mathbb{R}^{1 \times T}$, which undergoes normalization, patching, and embedding prior to being reprogrammed with learned text prototypes to align the source and target modalities. Then, we augment the LLM’s time series reasoning ability by prompting it together with reprogrammed patches to generate output representations, which are projected to the final forecasts $\hat{\mathbf{Y}}^{(i)} \in \mathbb{R}^{1 \times H}$.

We note that only the parameters of the lightweight input transformation and output projection are updated, while the backbone language model is frozen. In contrast to vision-language and other multimodal language models, which usually fine-tune with paired cross-modality data, TIME-LLM is directly optimized and becomes readily available with only a small set of time series and a few training epochs, maintaining high efficiency and imposing fewer resource constraints compared to building large domain-specific models from scratch or fine-tuning them. To further reduce memory footprints, various off-the-shelf techniques (e.g., quantization) can be seamlessly integrated for slimming TIME-LLM.

3.1 MODEL STRUCTURE

Input Embedding. Each input channel $\mathbf{X}^{(i)}$ is first individually normalized to have zero mean and unit standard deviation via reversible instance normalization (RevIN) in mitigating the time series distribution shift (Kim et al., 2021). Then, we divide $\mathbf{X}^{(i)}$ into several consecutive overlapped or non-overlapped patches (Nie et al., 2023) with length L_p ; thus the total number of input patches is $P = \lfloor \frac{T-L_p}{S} \rfloor + 2$, where S denotes the horizontal sliding stride. The underlying motivations are two-fold: (1) better preserving local semantic information by aggregating local information into each patch and (2) serving as tokenization to form a compact sequence of input tokens, reducing computational burdens. Given these patches $\mathbf{X}_P^{(i)} \in \mathbb{R}^{P \times L_p}$, we embed them as $\hat{\mathbf{X}}_P^{(i)} \in \mathbb{R}^{P \times d_m}$, adopting a simple linear layer as the patch embedder to create dimensions d_m .

Patch Reprogramming. Here we reprogram patch embeddings into the source data representation space to align the modalities of time series and natural language to activate the backbone’s time series understanding and reasoning capabilities. A common practice is learning a form of “noise” that, when applied to target input samples, allows the pre-trained source model to produce the desired target outputs without requiring parameter updates. This is technically feasible for bridging data

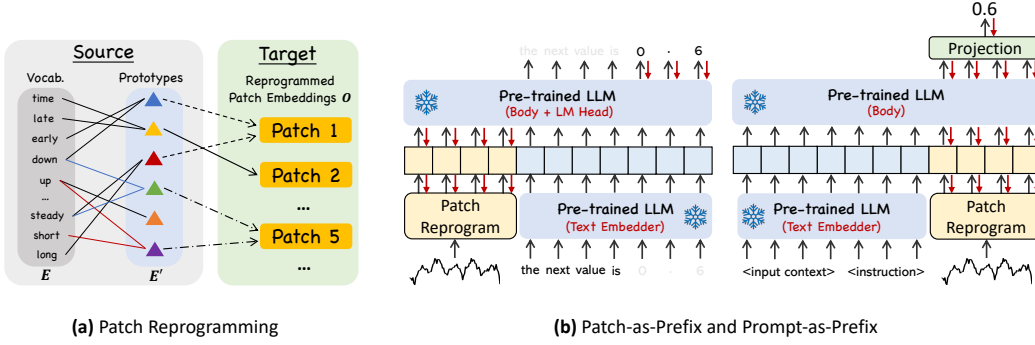


Figure 3: Illustration of (a) patch reprogramming and (b) Patch-as-Prefix versus Prompt-as-Prefix.

modalities that are identical or similar. Examples include repurposing a vision model to work with cross-domain images (Misra et al., 2023) or reprogramming an acoustic model to handle time series data (Yang et al., 2021). In both cases, there are explicit, learnable transformations between the source and target data, allowing for the direct editing of input samples. However, time series can neither be directly edited nor described losslessly in natural language, posing significant challenges to directly bootstrap the LLM for understanding time series without resource-intensive fine-tuning.

To close this gap, we propose reprogramming $\hat{\mathbf{X}}_P^{(i)}$ using pre-trained word embeddings $\mathbf{E} \in \mathbb{R}^{V \times D}$ in the backbone, where V is the vocabulary size. Nevertheless, there is no prior knowledge indicating which source tokens are directly relevant. Thus, simply leveraging $\mathbf{E}$ will result in large and potentially dense reprogramming space. A simple solution is to maintain a small collection of text prototypes by linearly probing $\mathbf{E}$, denoted as $\mathbf{E}' \in \mathbb{R}^{V' \times D}$, where $V' \ll V$. An illustration is in Fig. 3(a). Text prototypes learn connecting language cues, e.g., “short up” (red lines) and “steady down” (blue lines), which are then combined to represent the local patch information (e.g., “short up then down steadily” for characterizing patch 5) without leaving the space where the language model is pre-trained. This approach is efficient and allows for the adaptive selection of relevant source information. To realize this, we employ a multi-head cross-attention layer. Specifically, for each head $k = \{1, \dots, K\}$, we define query matrices $\mathbf{Q}_k^{(i)} = \hat{\mathbf{X}}_P^{(i)} \mathbf{W}_k^Q$, key matrices $\mathbf{K}_k^{(i)} = \mathbf{E}' \mathbf{W}_k^K$, and value matrices $\mathbf{V}_k^{(i)} = \mathbf{E}' \mathbf{W}_k^V$, where $\mathbf{W}_k^Q \in \mathbb{R}^{d_m \times d}$ and $\mathbf{W}_k^K, \mathbf{W}_k^V \in \mathbb{R}^{D \times d}$. Specifically, D is the hidden dimension of the backbone model, and $d = \lfloor \frac{d_m}{K} \rfloor$. Then, we have the operation to reprogram time series patches in each attention head defined as:

$$\mathbf{Z}_k^{(i)} = \text{ATTENTION}(\mathbf{Q}_k^{(i)}, \mathbf{K}_k^{(i)}, \mathbf{V}_k^{(i)}) = \text{SOFTMAX}\left(\frac{\mathbf{Q}_k^{(i)} \mathbf{K}_k^{(i)\top}}{\sqrt{d_k}}\right) \mathbf{V}_k^{(i)}. \quad (1)$$

By aggregating each $\mathbf{Z}_k^{(i)} \in \mathbb{R}^{P \times d}$ in every head, we obtain $\mathbf{Z}^{(i)} \in \mathbb{R}^{P \times d_m}$. This is then linearly projected to align the hidden dimensions with the backbone model, yielding $\mathbf{O}^{(i)} \in \mathbb{R}^{P \times D}$.

Prompt-as-Prefix. Prompting serves as a straightforward yet effective approach task-specific activation of LLMs (Yin et al., 2023). However, the direct translation of time series into natural language presents considerable challenges, hindering both the creation of instruction-following datasets and the effective utilization of on-the-fly prompting without performance compromise (Xue & Salim, 2022). Recent advancements indicate that other data modalities, such as images, can be seamlessly integrated as the prefixes of prompts, thereby facilitating effective reasoning based on these inputs (Tsimpoukelli et al., 2021). Motivated by these findings, and to render our approach directly applicable to real-world time series, we pose an alternative question: *can prompts act as pre-fixes to enrich the input context and guide the transformation of reprogrammed time series patches?* We term this concept as *Prompt-as-Prefix* (PaP) and observe that it significantly enhances the LLM’s adaptability to downstream tasks while complementing patch reprogramming (See Sec. 4.5 later).

The Electricity Transformer Temperature (ETT) indicates the electric power long-term deployment. Each data point consists of the target oil temperature and 6 power load features ...
Below is the information about the input time series:

[BEGIN DATA]

[Domain]: We usually observe that electricity consumption peaks at noon, with a significant increase in transformer load

[Instruction]: Predict the next <H> steps given the previous <T> steps information attached

[Statistics]: The input has a minimum of <min_val>, a maximum of <max_val>, and a median of <median_val>. The overall trend is <upward or downward>. The top five lags are <lag_val>.
[END DATA]

Figure 4: Prompt example. <> and <> are task-specific configurations and calculated input statistics.

An illustration of the two prompting approaches is in Fig. 3(b). In *Patch-as-Prefix*, a language model is prompted to predict subsequent values in a time series, articulated in natural language. This approach encounters certain constraints: (1) language models typically exhibit reduced sensitivity in processing high-precision numerals without the aid of external tools, thereby presenting substantial challenges in accurately addressing practical forecasting tasks over long horizons; (2) intricate, customized post-processing is required for different language models, given that they are pre-trained on diverse corpora and may employ different tokenization types in generating high-precision numerals with precision and efficiency. This results in forecasts being represented in disparate natural language formats, such as ['0', '.', '6', '1'] and ['0', '.', '61'], to denote the decimal 0.61.

Prompt-as-Prefix, on the other hand, tactfully avoids these constraints. In practice, we identify three pivotal components for constructing effective prompts: (1) dataset context, (2) task instruction, and (3) input statistics. A prompt example is in Fig. 4. The dataset context furnishes the LLM with essential background information concerning the input time series, which often exhibits distinct characteristics across various domains. Task instruction serves as a crucial guide for the LLM in the transformation of patch embeddings for specific tasks. We also enrich the input time series with additional crucial statistics, such as trends and lags, to facilitate pattern recognition and reasoning.

Output Projection. Upon packing and feedforwarding the prompt and patch embeddings $\mathbf{O}^{(i)}$ through the frozen LLM as shown in Fig. 2, we discard the prefixal part and obtain the output representations. Following this, we flatten and linear project them to derive the final forecasts $\hat{\mathbf{Y}}^{(i)}$.

4 MAIN RESULTS

TIME-LLM consistently outperforms state-of-the-art forecasting methods by large margins across multiple benchmarks and settings, especially in few-shot and zero-shot scenarios. We compared our approach against a broad collection of up-to-date models, including a recent study that fine-tunes language model for time series analysis (Zhou et al., 2023a). To ensure a fair comparison, we adhere to the experimental configurations in (Wu et al., 2023) across all baselines with a unified evaluation pipeline¹. We use Llama-7B (Touvron et al., 2023) as the default backbone unless stated otherwise.

Baselines. We compare with the SOTA time series models, and we cite their performance from (Zhou et al., 2023a) if applicable. Our baselines include a series of Transformer-based methods: PatchTST (2023), ESTformer (2022), Non-Stationary Transformer (2022), FEDformer (2022), Autoformer (2021), Informer (2021), and Reformer (2020). We also select a set of recent competitive models, including GPT4TS (2023a), LLMTime (2023), DLinear (2023), TimesNet (2023), and LightTS (2022a). In short-term forecasting, we further compare our model with N-HiTS (2023b) and N-BEATS (2020). More details are in Appendix A.

4.1 LONG-TERM FORECASTING

Setups. We evaluate on ETTh1, ETTh2, ETTm1, ETTm2, Weather, Electricity (ECL), Traffic, and ILI, which have been extensively adopted for benchmarking long-term forecasting models (Wu et al., 2023). Details of the implementation and datasets can be found in Appendix B. The input time series length T is set as 512, and we use four different prediction horizons $H \in \{96, 192, 336, 720\}$. The evaluation metrics include mean square error (MSE) and mean absolute error (MAE).

Results. Our brief results are shown in Tab. 1, where TIME-LLM outperforms all baselines in most cases and significantly so to the majority of them. The comparison with GPT4TS (Zhou et al., 2023a) is particularly noteworthy. GPT4TS is a very recent work that involves fine-tuning on the backbone language model. We note average performance gains of **12%** and **20%** over GPT4TS and TimesNet, respectively. When compared with the SOTA task-specific Transformer model PatchTST, by reprogramming the smallest Llama, TIME-LLM realizes an average MSE reduction of 1.4%. Relative to the other models, e.g., DLinear, our improvements are also pronounced, exceeding **12%**.

4.2 SHORT-TERM FORECASTING

Setups. We choose the M4 benchmark (Makridakis et al., 2018) as the testbed, which contains a collection of marketing data in different sampling frequencies. More details are provided in Appendix B. The prediction horizons in this case are relatively small and in [6, 48]. The input lengths

¹<https://github.com/thuml/Time-Series-Library>